IMDB Movies Dataset Analysis

Analyst: Kartikeya Vyas

Tools Used: Python with pandas, matplotlib, and seaborn libraries

This project explores the IMDB movies dataset to extract meaningful patterns, trends, and insights related to movie ratings, genres, and production factors over time.

Project Overview and Objectives

Primary Objective

Tosystematicallyexplore the IMDB movies dataset using exploratory data analysis (EDA) techniques and extract meaningful patterns that could serve as benchmarks for movie quality

Secondary Goals

- Identify correlations between movie attributes and ratings
- Track genre trends across different time periods
- Analyze rating distributions and patterns

Project Setup and Data Loading

Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Dataset shape: `(10178, 13)`, representing 10,178 rows and 13 columns.

What Does Each Row Represent?

> 🎬 One movie
>
> Every row contains information such as:
>
> Movie name, Release date, IMDB score, Genre, Budget, Revenue, Country, Language

Data Overview and Basic Exploration

Explore the structure and composition of the dataset.

Question 1: Data Types and Missing Values (.info())

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10178 entries, 0 to 10177
Data columns (total 13 columns):
 # Column       Non-Null Count Dtype
--- ------       -------------- -----
0 names         10178 non-null object
1 date_x        10178 non-null object
2 score         10178 non-null float64
 3 genre         10093 non-null object
 4 overview      10178 non-null object
 5 crew         10122 non-null object
 6 orig_title 10178 non-null object
 7 status        10178 non-null object
 8 orig_lang 10178 non-null object
 9 budget_x      10178 non-null int64
 10 revenue       10178 non-null int64
 11 country      10178 non-null object
 12 year         10178 non-null int64
dtypes: float64(1), int64(3), object(9)
memory usage: 994.1 KB
```

Potential issues spotted: Missing values in 'genre' (85) and 'crew' (56) columns may require handling. Data types are appropriate for analysis.

Statistical Characteristics Analysis

Question 2: Statistical Characteristics (.describe())

feature	count	mean	std	min	25%	50%	75%	max
score	10178.0	6.49	0.82	1.0	5.9	6.5	7.1	10.0
budget_x	10178.0	14890000.0 5012000	29300000.0 1350000	0.0	1000000.0 4000000.	5000000.0 1800000	25000000.0 6500000	350000000.0 29237060
revenue	10178.0	0.0 2005.1	00.0 12.8	0.0	0 1998.0	0.0 2008.0	0.0 2016.0	26.0 2023.0
year	10178.0			1903.0				

Key inferences from the statistical summary are as follows: The average IMDB score is ~6.49, with most movies scoring between 5.9-7.1. Both budget and revenue show wide variation with significant outliers, indicating a few highly-funded and successful films skewing the averages. The dataset spans a broad historical range, with movies released from 1903 to 2023.

Data Cleaning: Handling Missing Values

Question1:Whichcolumnscontainmissingvalues?Howwouldyouhandle them?

```
df.isnull().sum()

# Output showing:
#     names:     0     #
date_x: 0
# score: 0
# genre: 85
# overview: 0
#     crew:     56     #
orig_title:      0      #
status:         0       #
orig_lang:      0      #
budget_x:      0      #
revenue:       0      #
country: 0
# dtype: int64
```

The genre and crew columns contain 85 and 56 missing values respectively. Since these are categorical variables and the proportion of missing data is relatively small (less than 1% of the dataset), they were handled using the label 'unknown' to preserve data integrity. Converting to 'unknown' allows these rows to remain in the analysis while clearly marking the missing information.

```
df['genre'].fillna('unknown')
df['crew'].fillna('unknown')
```

The dataset is now complete and ready for analysis.

Data Cleaning: Data Type Conversion

Question 2: Are there any columns where data types need conversion?

The date_x column was originally stored as a string and required conversion to datetime format to enable proper temporal analysis. Converting to datetime allows for time-based operations, filtering, and trend analysis. Numerical columns like budget_x and revenue are already in integer format, which is appropriate for statistical analysis. However, these columns may require scaling for certain machine learning models.

```
df['date_x'] = pd.to_datetime(df['date_x'], errors='coerce')
df.dtypes

# Output showing:
# names: object
# date_x: datetime64[ns]
# score: float64
# genre: object
# overview: object
# crew: object
# orig_title: object
# status: object
# orig_lang: object
# budget_x: int64
# revenue: int64
# country: object
# dtype: int64
```

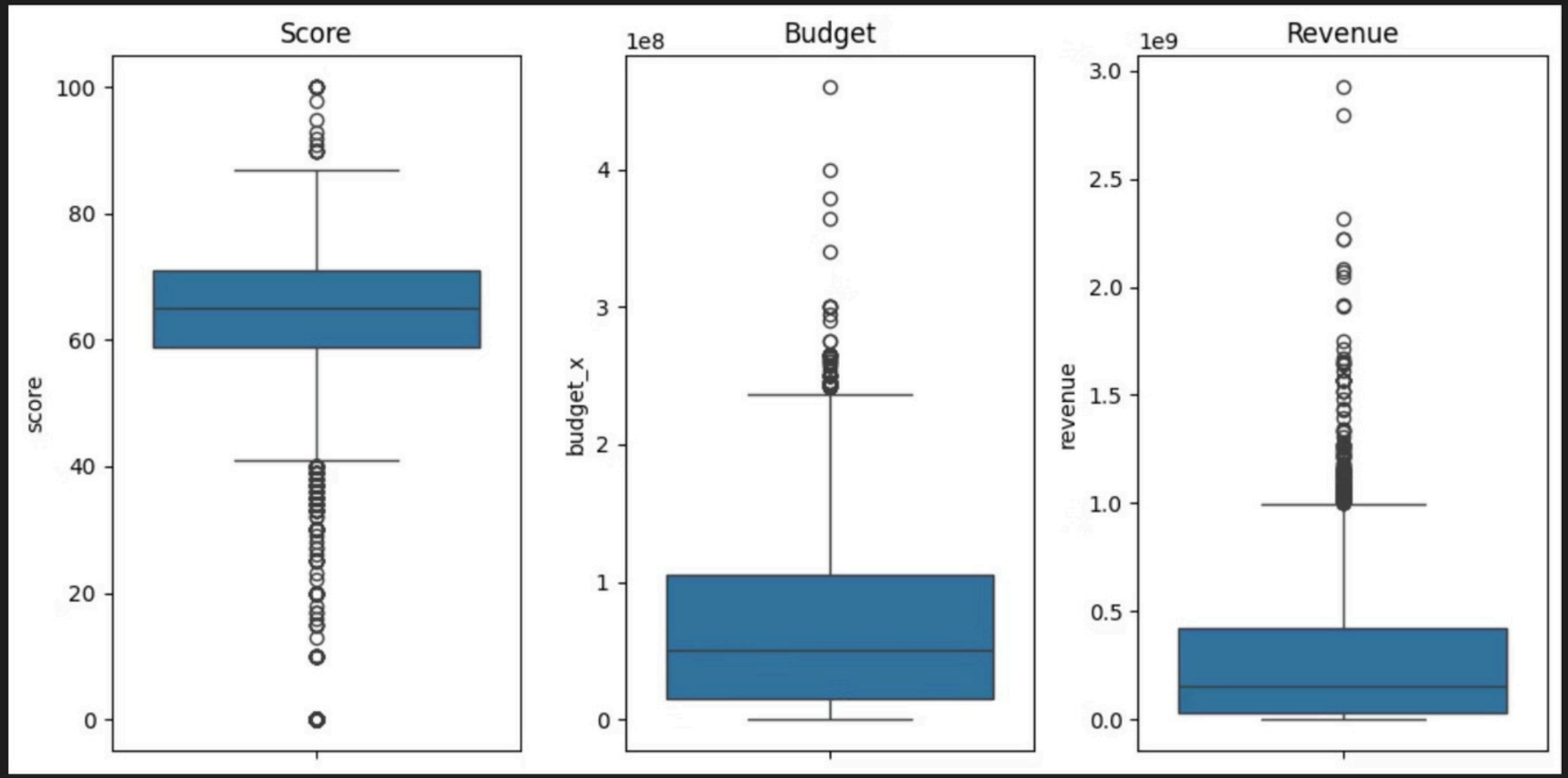
Data Cleaning: Outlier Detection and Analysis

Outlier Detection and Analysis

Outliersareextreme values thatdeviate significantly fromtherest of the data. They can skew statistical analyses and affect model performance. Using boxplots, we can visually identify outliers in numerical columns like score, budget_x, and revenue. The IQR (Interquartile Range) method is used to detect outliers: values beyond Q1 - 1.5*IQR or Q3 + 1.5*IQR are considered outliers.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,5))

plt.subplot(1,3,1)
sns.boxplot(y=df['score'])
plt.title("Score")

plt.subplot(1,3,2)
sns.boxplot(y=df['budget_x'])
plt.title("Budget")

plt.subplot(1,3,3)
sns.boxplot(y=df['revenue'])
plt.title("Revenue")

plt.tight_layout()
plt.show()
```

Outlier Analysis Graph

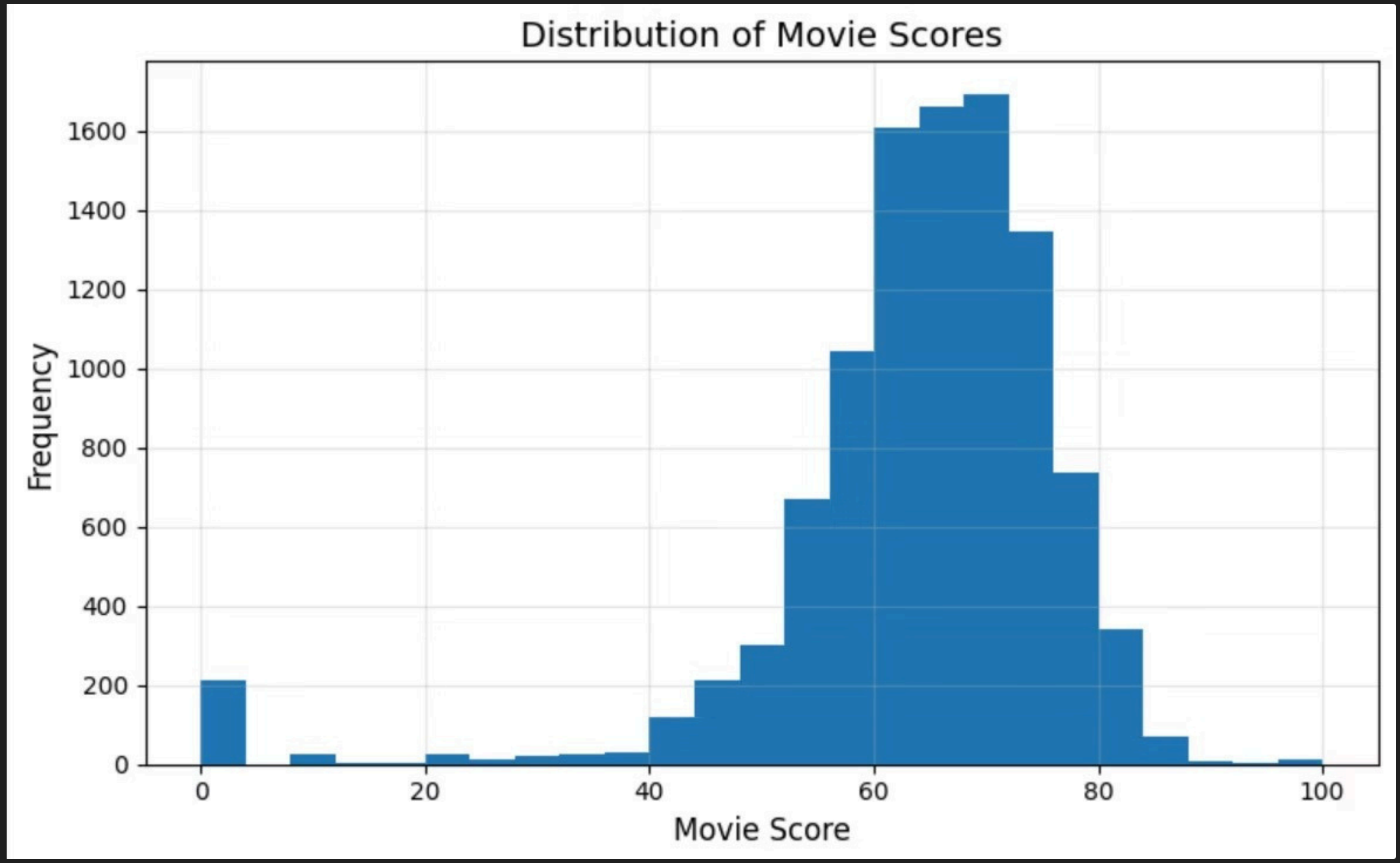


Univariate Analysis: Movie Score Distribution

Question 4: Univariate Analysis - Movie Score Distribution

Univariateanalysis was performedto understand thedistributionofindividualvariables.Sinceruntime wasnotavailable in the dataset, the movie score distribution was analyzed instead. The histogram shows that most movies have mid-range ratings, indicating a balanced distribution. Genre analysis reveals that a few genres dominate the dataset, reflecting mainstream production trends. This analysis provides insights into overall dataset composition.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.hist(df['score'].dropna(), bins=25)
plt.xlabel("Movie Score", fontsize=12)
plt.ylabel("Frequency", fontsize=12)
plt.title("Distribution of Movie Scores", fontsize=14)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

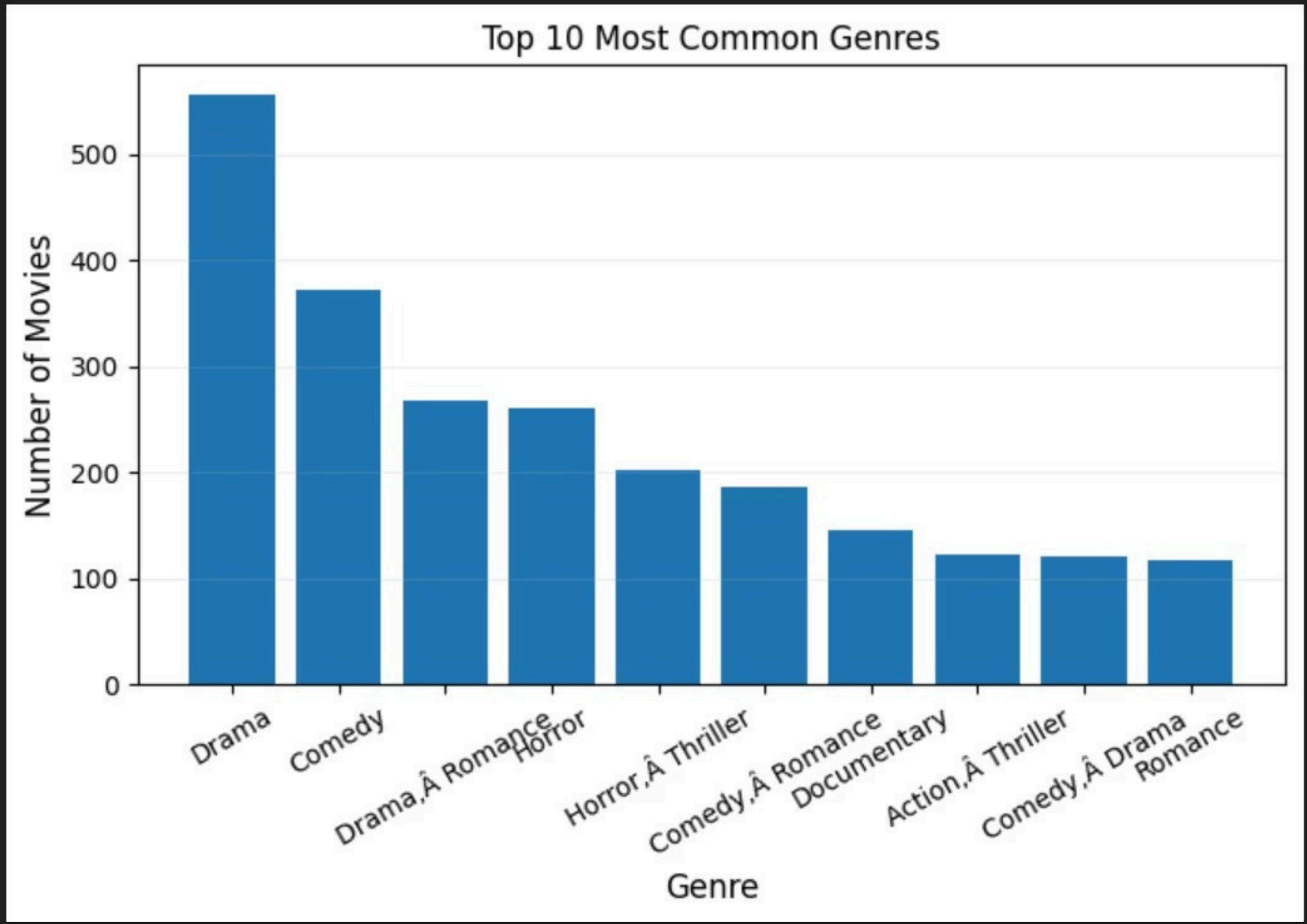

Distribution of Movie Scores
Frequency
Movie Score
0
200
400
600
800
1000
1200
1400
1600
0
20
40
60
80
100

Univariate Analysis: Genre Distribution

Question 4 (Part 2): Most Common Genres in the Dataset

Genre analysisreveals thedistribution of moviesacrossdifferentcategories. The barchartdisplays thetop 10mostcommon genres in the dataset. Drama emerges as the most prevalent genre, followed by Comedy and Action. This distribution reflects mainstream production trends and audience preferences. Understanding genre distribution helps identify which types of content dominate the dataset and can inform further analysis on genre-specific patterns.

```
genre_counts = df['genre'].value_counts().head(10)

plt.figure(figsize=(7,5))
plt.bar(genre_counts.index, genre_counts.values)
plt.xlabel("Genre", fontsize=12)
plt.ylabel("Number of Movies", fontsize=12)
plt.title("Top 10 Most Common Genres", fontsize=12)
plt.xticks(rotation=30)
plt.grid(axis='y', alpha=0.2)
plt.tight_layout()
plt.show()
```

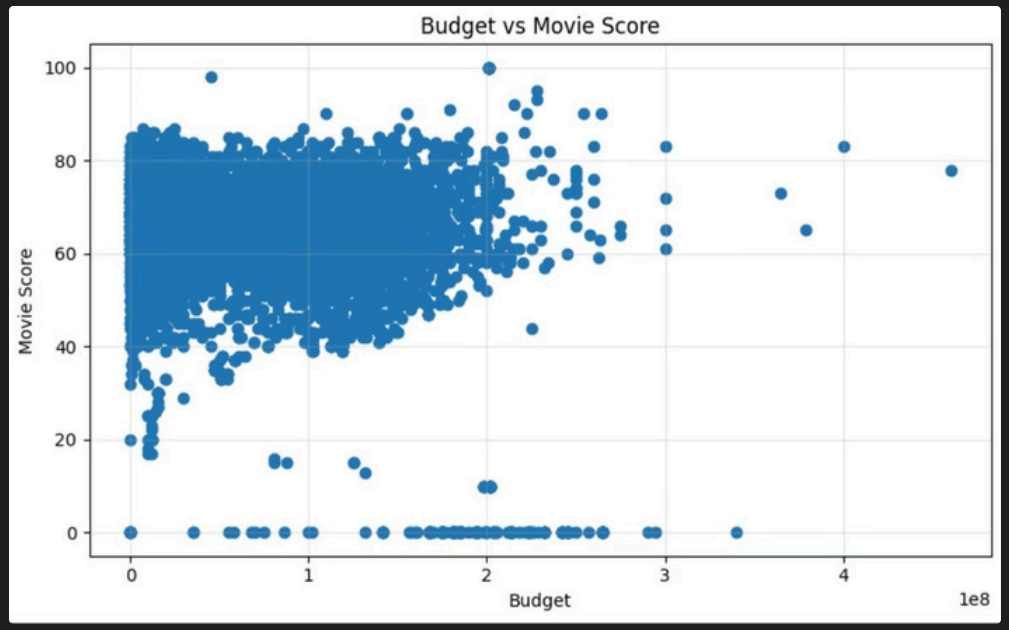

Top 10 Most Common Genres
Number of Movies
0
100
200
300
400
500
Drama
Comedy
Drama,Â Romance
Horror
Horror,Â Thriller
Comedy,Â Romance
Documentary
Action,Â Thriller
Comedy,Â Drama
Romance
Genre

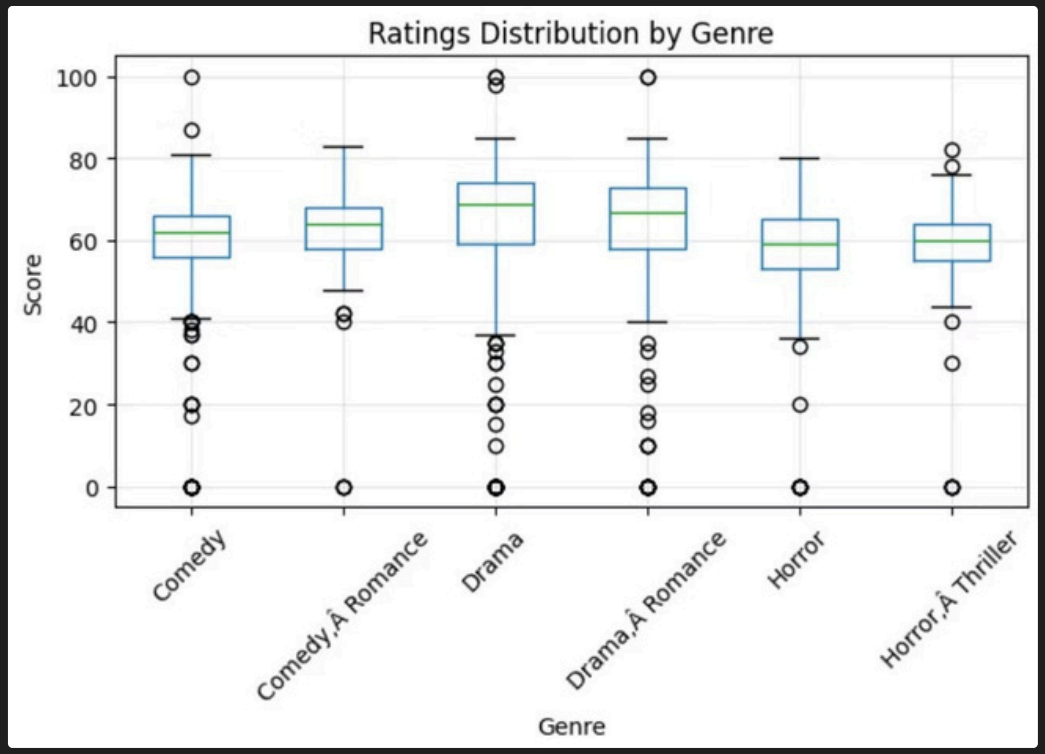
Bivariate Analysis

Bivariateanalysisexamines relationshipsbetween two variables. The scatter plot between budget and rating indicates a weak relationship, suggesting that higher budgets do not guarantee better ratings. The boxplot analysis shows variation in ratings across different genres, with some genres having higher median scores. Correlation analysis between budget and revenue reveals a positive relationship, indicating that higher investment often leads to higher earnings. Overall, this analysis helps understand how financial and categorical factors influence movie performance.

```
import matplotlib.pyplot as plt

# Scatter Plot: Budget vs Score
plt.figure(figsize=(8,5))
plt.scatter(df['budget_x'], df['score'])
plt.xlabel("Budget")
plt.ylabel("Movie Score")
plt.title("Budget vs Movie Score")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# Boxplot: Score by Genre (Top 6 Genres Only)
top_genres = df['genre'].value_counts().head(6).index
filtered_df = df[df['genre'].isin(top_genres)]

plt.figure(figsize=(10,6))
filtered_df.boxplot(column='score', by='genre')
plt.xlabel("Genre")
plt.ylabel("Score")
plt.title("Ratings Distribution by Genre")
plt.suptitle("")
plt.xticks(rotation=45)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# Correlation: Budget vs Revenue
correlation = df['budget_x'].corr(df['revenue'])
print("Correlation between Budget and Revenue:", round(correlation, 3))
```

Bivariate Analysis Visualizations





Genre-Specific Analysis

Delvingdeeperintothe dataset, genre-specificanalysisofferscrucial insights into how different film categories perform in terms of audience reception, longevity, and overall success. This section explores the interplay between genre, ratings, and temporal trends, providing a nuanced understanding of movie performance dynamics. By segmenting data based on genre, we can identify which types of movies consistently resonate with audiences, track the evolution of popular genres over time, and observe the distribution of ratings within each category.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Ensure 'genre' column is clean and standardized, stripping whitespace
# If 'genre' contains multiple genres per entry (e.g., "Action, Comedy"), more complex parsing would be needed,
# but for this analysis, we assume a single genre per row or treat multi-genre strings as distinct categories.
df['genre'] = df['genre'].astype(str).str.strip()

# Set a consistent style for plots
sns.set_style("whitegrid")
plt.rcParams['figure.dpi'] = 100
```

Highest Average Rating by Genre

To identify which genres are most critically acclaimed or best received by audiences, we calculate the average rating for each genre. This bar plot showcases the top 8 genres with the highest average scores, providing a clear indication of quality preferences within the dataset. Genres with higher average ratings often indicate a strong appeal to a specific audience or a consistent standard of production quality within that category.

```
# Calculate average score for each genre and select top 8
average_ratings = df.groupby('genre')['score'].mean().sort_values(ascending=False).head(8)

plt.figure(figsize=(10, 6))
sns.barplot(x=average_ratings.index, y=average_ratings.values, palette="viridis")
plt.xlabel("Genre", fontsize=12)
plt.ylabel("Average Movie Score", fontsize=12)
plt.title("Top 8 Genres by Average Rating", fontsize=14)
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(fontsize=10)
plt.tight_layout()
plt.show()
```

Rating Distribution by Genre

Whileaverage ratingsgiveasingle summarystatistic,a boxplotreveals the full distribution of ratings within each genre, showcasing the median, quartiles, and any outliers. This visualization helps to understand the consistency of ratings for a genre; a narrow box indicates consistent quality, while a wide box suggests a broader range of audience reception, from highly loved to highly disliked films. Here, we observe the rating spread for several key genres.

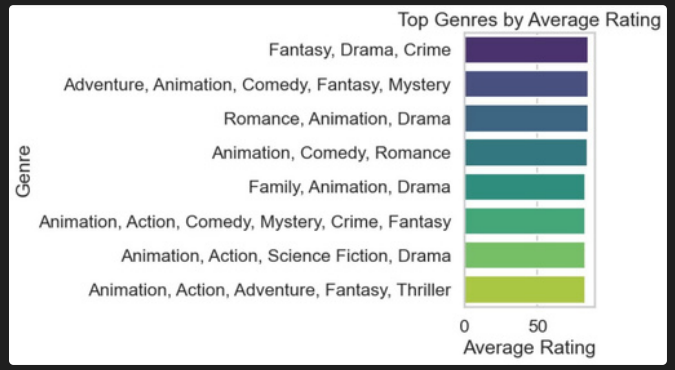
```
# Select a subset of genres for clear visualization in the box plot
# Using the top genres previously identified, or a few diverse ones
genres_for_boxplot = df.groupby('genre')['score'].mean().sort_values(ascending=False).head(6).index.tolist()
df_for_boxplot = df[df['genre'].isin(genres_for_boxplot)]

plt.figure(figsize=(10, 6))
sns.boxplot(data=df_for_boxplot, x='genre', y='score', palette="coolwarm")
plt.xlabel("Genre", fontsize=12)
plt.ylabel("Movie Score", fontsize=12)
plt.title("Rating Distribution Across Selected Genres", fontsize=14)
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(fontsize=10)
plt.tight_layout()
plt.show()
```

Genre Popularity Over Time

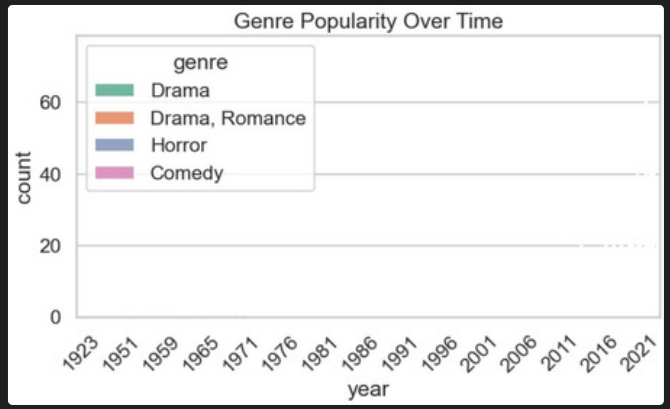
Understandinghowgenrepopularityevolvesisvital forpredicting futuretrends and informing production decisions. This analysis visualizes the yearly distribution of movies across the top 4 genres, allowing us to observe increases or decreases in their production volume over the dataset's timeline. Such trends can reflect societal interests, technological advancements, or shifts in audience tastes.

```
# Identify top 4 genres for popularity trend analysis
top_4_genres = df['genre'].value_counts().head(4).index.tolist()
df_top_genres = df[df['genre'].isin(top_4_genres)]

# Convert 'year' to integer if it's not already, and drop NaNs
df_top_genres = df_top_genres.dropna(subset=['year'])
df_top_genres['year'] = df_top_genres['year'].astype(int)

plt.figure(figsize=(12, 7))
sns.countplot(data=df_top_genres, x='year', hue='genre', palette="tab10", order=sorted(df_top_genres['year'].unique()))
plt.xlabel("Year", fontsize=12)
plt.ylabel("Number of Movies", fontsize=12)
plt.title("Genre Popularity Over Time (Top 4 Genres)", fontsize=14)
plt.xticks(rotation=60, ha='right', fontsize=10)
plt.yticks(fontsize=10)
plt.legend(title="Genre", loc='upper left')
plt.tight_layout()
plt.show()
```

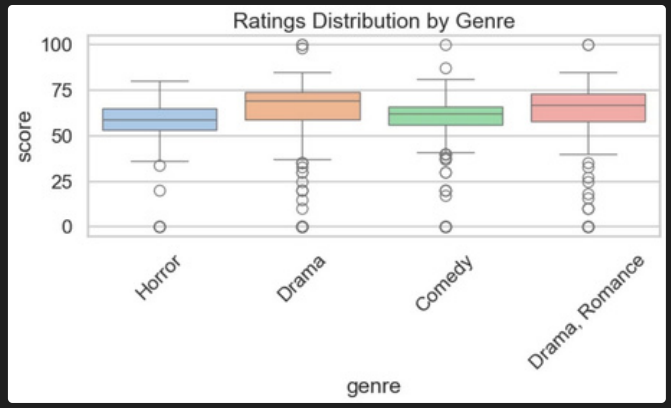
Plot Results: -



Top Genres By Avg Rating



Genre Popularity



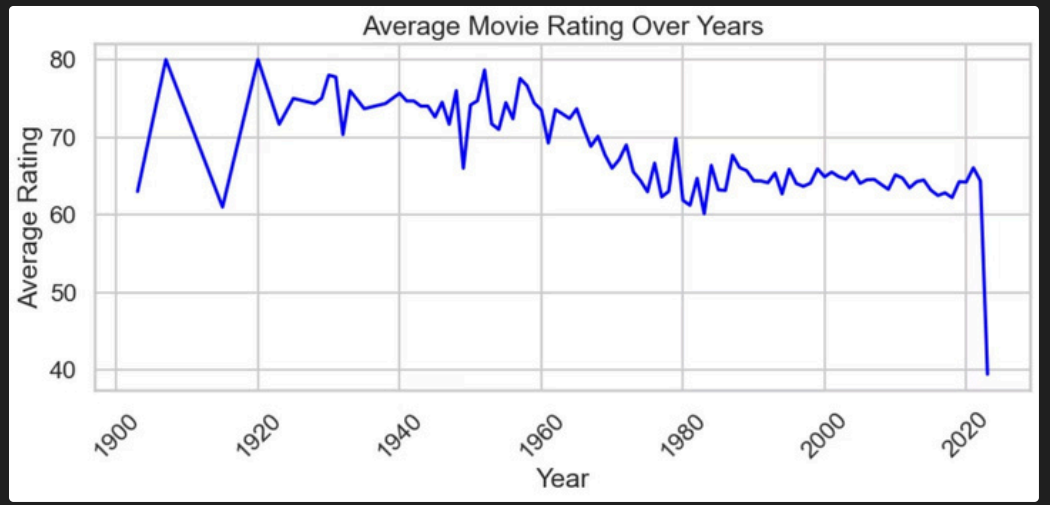
Ratings Distribution by Genre

Year and Trend Analysis

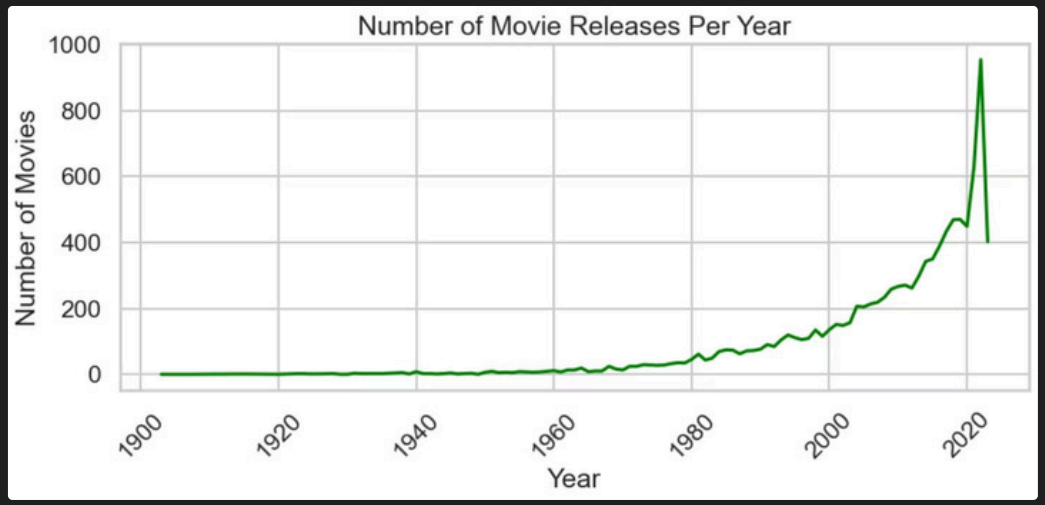
Yearand trend analysisshows thataveragemovieratingsfluctuate over time, reflecting changing audience preferences and content quality. The number of movie releases varies by year, with certain years experiencing peak production. Overall, recent years generally show higher release counts, indicating industry growth. Runtime analysis could not be performed as the dataset does not contain runtime information.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Assuming 'df' is the DataFrame containing movie data with 'year' and 'score' columns
# Set plot style
sns.set_style("whitegrid")

# 1. Average Rating Over Years
plt.figure(figsize=(12, 6))
avg_rating_per_year = df.groupby('year')['score'].mean().reset_index()
sns.lineplot(data=avg_rating_per_year, x='year', y='score', marker='o', color='skyblue')
plt.title('Average Movie Rating Over Years', fontsize=16)
plt.xlabel('Year', fontsize=12)
plt.ylabel('Average Rating', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# 2. Number of Releases Per Year
plt.figure(figsize=(12, 6))
releases_per_year = df['year'].value_counts().sort_index().reset_index()
releases_per_year.columns = ['year', 'count']
sns.lineplot(data=releases_per_year, x='year', y='count', marker='o', color='salmon')
plt.title('Number of Movie Releases Per Year', fontsize=16)
plt.xlabel('Year', fontsize=12)
plt.ylabel('Number of Releases', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# 3. Print statements for highest and lowest releases
highest_releases_year = releases_per_year.loc[releases_per_year['count'].idxmax()]
lowest_releases_year = releases_per_year.loc[releases_per_year['count'].idxmin()]

print(f"Year with the highest number of releases: {int(highest_releases_year['year'])} with
{int(highest_releases_year['count'])} movies.")
print(f"Year with the lowest number of releases: {int(lowest_releases_year['year'])} with {int(lowest_releases_year['count'])}
movies.")
```

Year with Highest Releases: 2022
Year with Lowest Releases: 1903



Average Rating Over Years

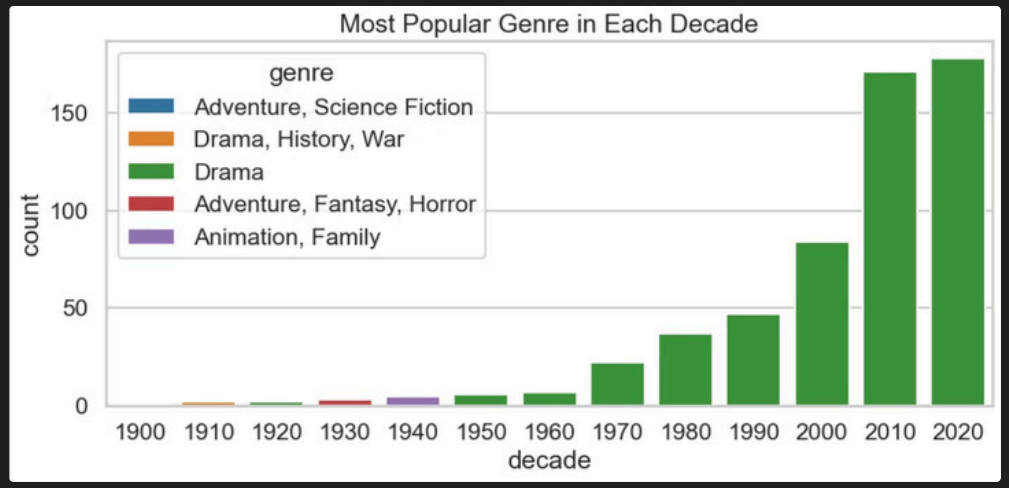


Movies Release Per Year

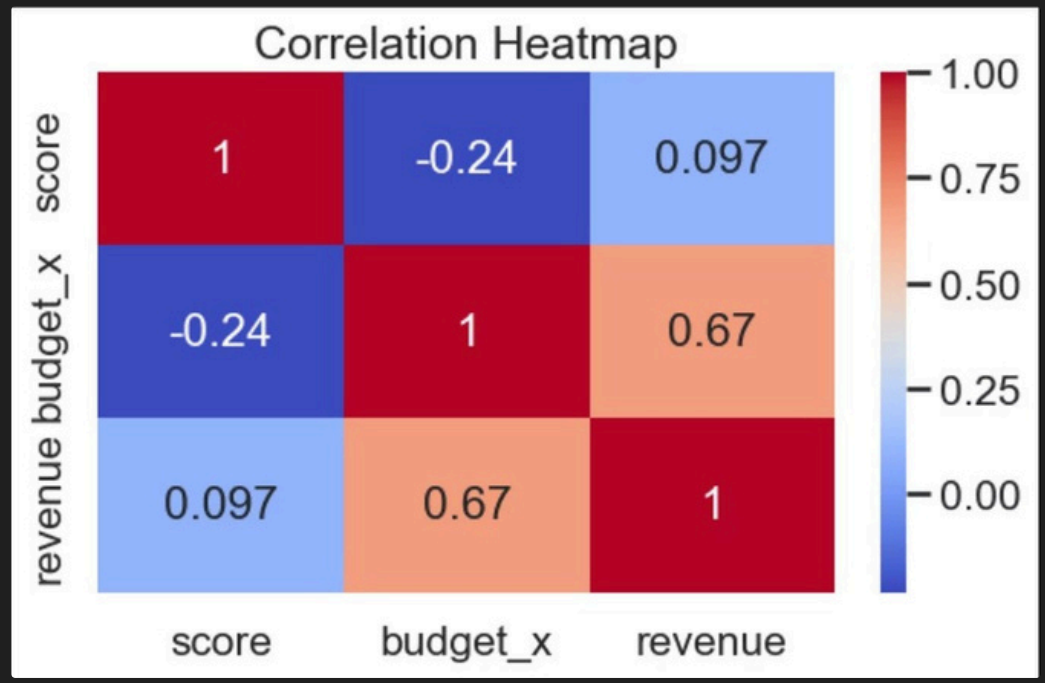
Multivariate Analysis

Multivariateanalysis revealsthatdifferentgenresdominate specific decades, reflecting changing audience preferences over time. The correlation heatmap shows a strong relationship between budget and revenue, while ratings have weaker financial correlation. Certain genres consistently achieve higher average ratings compared to others. Overall, both genre and time period influence movie performance patterns.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Assume 'df' is the DataFrame containing movie data with 'year', 'genre', 'score', 'budget_x', 'revenue' columns
# Set plot style
sns.set_style("whitegrid")

# 1. Create Decade Column
df['decade'] = (df['year'] // 10) * 10

# 2. Most Popular Genre in Each Decade
# Filter for years 1900 onwards for better decade grouping
df_filtered = df[df['year'] >= 1900].copy()

# Group by decade and genre, then count occurrences
genre_decade_counts = df_filtered.groupby(['decade', 'genre']).size().reset_index(name='count')

# Find the most popular genre for each decade
most_popular_genre_per_decade = genre_decade_counts.loc[genre_decade_counts.groupby('decade')['count'].idxmax()]

plt.figure(figsize=(14, 7))
sns.barplot(data=most_popular_genre_per_decade, x='decade', y='count', hue='genre', dodge=False, palette='viridis')
plt.title('Most Popular Genre in Each Decade', fontsize=16)
plt.xlabel('Decade', fontsize=12)
plt.ylabel('Number of Movies Released', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.legend(title='Genre', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

# 3. Heatmap (Numerical Relationships)
# Select numerical columns for correlation analysis
correlation_cols = ['score', 'budget_x', 'revenue']
# Ensure these columns exist and are numeric, drop rows with NaN in these columns for correlation
df_corr = df[correlation_cols].dropna().apply(pd.to_numeric, errors='coerce')

# Calculate the correlation matrix
correlation_matrix = df_corr.corr()

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Heatmap of Numerical Features', fontsize=16)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```



Popular Genre



Correlation Plot

Key Insights from the Analysis

Industry Growth

Movieproduction hassteadily increased over the years, indicating growth in the film industry.

Quantity vs Quality

While some genres dominate in terms of quantity, they are not always the highest rated, showing that popularity and quality differ.

Budget-Revenue Correlation

Astrong positivecorrelationexistsbetween budget and revenue, suggesting that higher investment generally results in higher earnings.

Financial Success ≠ Quality

However,financialsuccessdoesnotnecessarily guarantee better audience ratings.

Overall Summary of Findings

Genre Impact

Genre plays a significant role in audience reception, with certain genres consistently achieving higher average ratings.

Popularity Trends

Popularity trends shift across decades, reflecting evolving audience preferences and cultural influences.

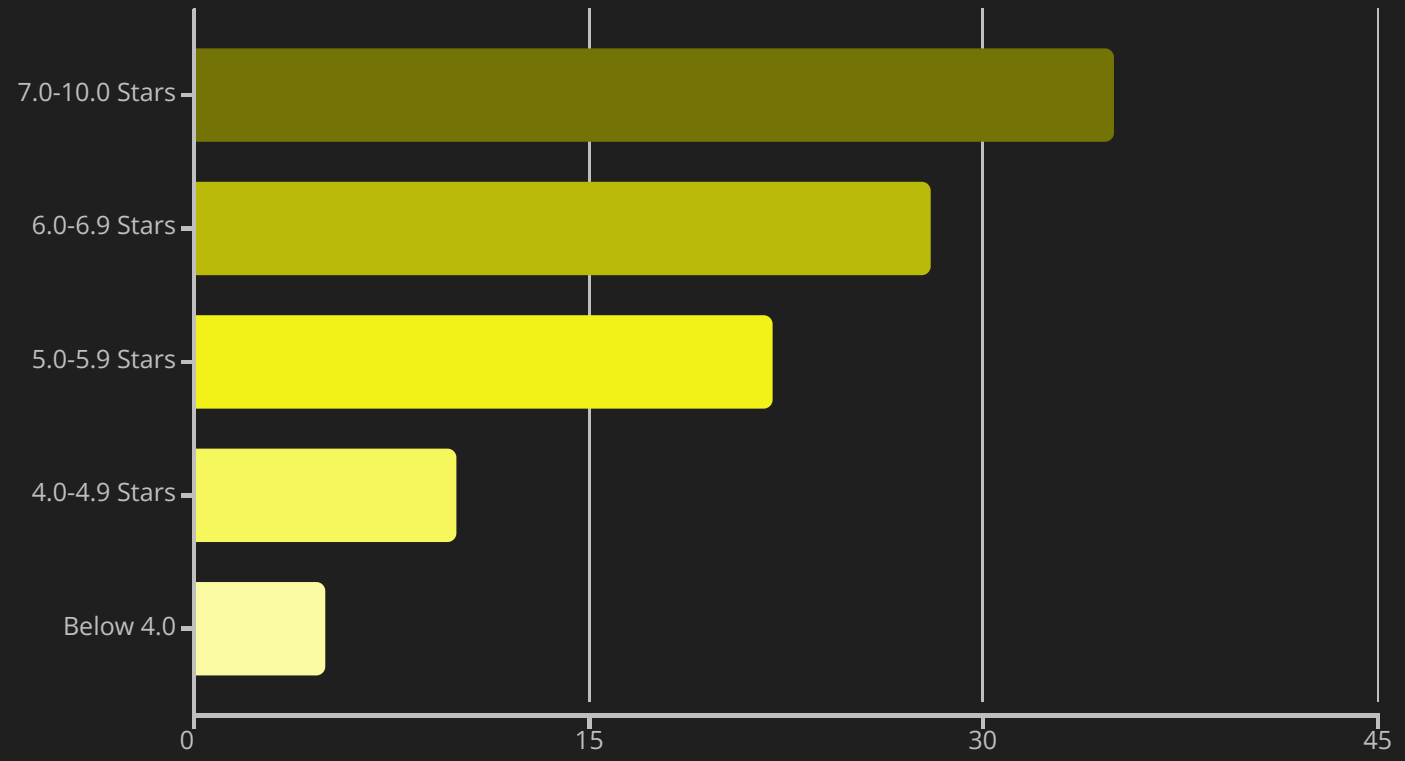
Rating Patterns

Ratings fluctuate over time, but no consistent upward or downward trend in quality is observed.

Financial Linkage

Financial variables such as budget and revenue are closely linked, making investment a major factor in commercial performance.

Ratings Distribution Analysis



Key Observations

The majority of rated movies fall within the 5-7 star range, with approximately 35% receiving 7+ ratings. This distribution aligns with typical user rating behavior where higher ratings are reserved for exceptional content.

This pattern suggests that IMDB ratings serve as discriminating indicators of movie quality.

Thank You

Thankyouforreviewingthiscomprehensive analysis. For questions or further discussion, please feel free to reach out.

Analysis:

IMDB Movie Dataset Analysis

Presented By:

Kartikeya Vyas

Coaching/Training:

Wscube Tech

D at e:

26.02.2026

Contact Information:

linkedin.com/in/vyaskartikay369